H-FARM INTERNATIONAL SCHOOL

HIS Management Platform — Complete Guide

H-FARM International School · Management Platform

Technical and operational guide for whoever runs the platform

Version of 20 October 2026 — up to date with database migration **0024**

Internal document · contains no passwords or keys

Contents

How to read this guide

Essential glossary

1. What the platform is

2. The overall picture

3. Where everything is

4. External platforms used

5. File structure

6. How the frontend works

7. Roles, permissions and companies

8. The backend: Supabase

9. What the database stores

10. The Edge Functions

11. Secrets and keys: where they are and what they are for

12. How the main flows work

13. Database migrations

14. How to publish a change

15. Maintenance and frequent operations

16. Troubleshooting

17. Final notes

How to read this guide

This guide explains **how the platform is built, where every piece lives and how to manage it**. It is written so that someone who is not a developer can follow it: every technical term is explained the first time it appears, and there is a glossary right below.

- For **the big picture**, read chapters 1, 2 and 3.
- To **make a change or publish it**, go to chapters 13 and 14.
- If **something is not working**, go to chapter 16.
- If you are looking for **a password, a key or a secret URL**, chapter 11 tells you where it is kept (the guide does not contain the values, only where they are stored).

> **Important:** this guide **does not contain passwords or secret keys**. It only says *where* they are stored and *what they are for*. Never add secret values to this document.

Essential glossary

Term	What it means, in plain words
Frontend	The part you see in the browser: pages, buttons, forms.
Backend	The "behind the scenes" part that stores data, checks who can do what and sends notifications. Here it is **Supabase**.
Database	The archive where data is stored, organised in **tables** (like Excel sheets with fixed columns). Here it is a **PostgreSQL** database inside Supabase.
Repository (repo)	The source-code folder, with the full history of its changes. It lives on **GitHub**.
Git / commit / push	Git is the tool that records code changes. *Commit* = save a group of changes with a description. *Push* = send them to GitHub.
Build	The "compilation": turns the source code into web pages ready to publish.
Deploy	Publishing the new version online.
Hosting	The service that "hosts" the site and makes it reachable from the internet. Here it is **Cloudflare Pages**.
Edge Function	A small program that runs on Supabase's servers (not in the browser). Used for operations that need secrets, e.g. sending data to Power Automate.
Secret	A confidential value (password, key, private URL) stored encrypted in the backend, never in the code.
Migration (SQL)	A `.sql` file that changes the database structure (creates tables, rules, functions). It is run by hand in the Supabase SQL Editor.
RLS (Row Level Security)	Database rules that decide, row by row, who may read or change data. Even if someone bypassed the pages, the database refuses anything not allowed.
RPC / SQL function	A function stored in the database that the platform calls for sensitive operations (e.g. changing a role): it checks permissions before acting.
Webhook	A web address (URL) a system sends data to in order to start an automation. Here: the **Power Automate** flows.

Term	What it means, in plain words
MFA	Two-factor authentication: besides the password, the 6-digit code from an app (Microsoft Authenticator, Google Authenticator...).
JWT / token	The digital "badge" the browser receives after login and shows to the backend with every request.
Anon key	Supabase public key used by the site. **It is not a secret**: security comes from the RLS rules.
Service role key	Supabase "administrator" key. **Top secret**: it must never be in the site or in the code.

1. What the platform is

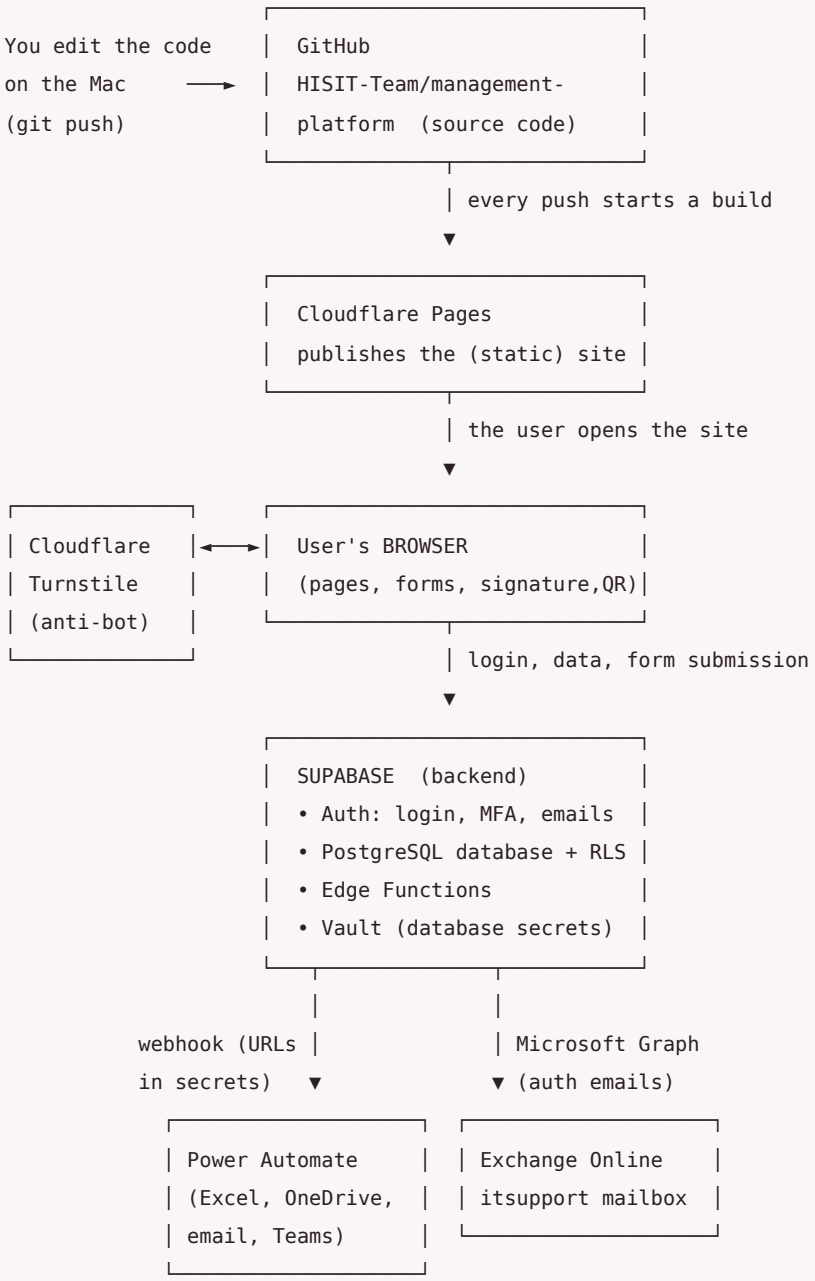
The **HIS Management Platform** is the internal portal of H-FARM International School. Each employee signs in with email and password (plus MFA if their role requires it) and sees **only the sections of their role**.

Main features active today:

Area	What you do there
Dashboard	Personalised home page: indicators, latest movements, budget, deadlines.
IT	Device delivery and return for students and staff (signature, photos, QR scanning), assignment history, **lost accessories**, registries, student QR cards, link to the Asset Manager.
Budget	Budget lines ("commesse") and expenses of the three schools (Venezia, Vicenza, Rosà), with new lines and allocation changes. *Owner, Super Admin, Admin only.*
Task Manager	IT team tasks with project groups, sub-tasks, list and kanban views, Teams notifications. *Owner, Super Admin, Admin only.*
HR	Onboarding (new employee) and offboarding (leaving employee) forms, registries.
Boarding	Room assignment, camera link.
Student Office	Area for the student office (forms coming soon).
Administration	Users and roles, permissions per role, activity log, security (MFA, timeout, remembered devices). *Owner and Super Admin only.*
Profile	Personal data, picture, email/password change, MFA, appearance preferences.

The platform is split into **three companies**: **H-IS Venezia**, **H-IS Vicenza**, **H-IS Rosà** (see chapter 7).

2. The overall picture



In short:

1. **The code** lives on GitHub.
2. **Cloudflare Pages** takes the code from GitHub, "builds" it and publishes the site.
3. **The site** runs in the browser and talks only to **Supabase**.
4. **Supabase** checks who you are, stores the data and, through the **Edge Functions**, forwards forms to **Power Automate** (which writes to Excel/OneDrive, sends emails and Teams messages).

The site **has no server of its own**: it is a set of static pages. All "confidential" logic lives in Supabase.

3. Where everything is

What	Where	Notes
Source code (official)	GitHub → organisation **HISIT-Team**, repository **management-platform** (`https://github.com/HISIT-Team/management-platform`)	This is the "real" version. Everything starts here.
Working copy on the Mac	`~/Downloads/management-platform/management-platform`	Changes are made here and pushed with `git push` .
Published site	**Cloudflare Pages**, connected to the GitHub repository	Updates by itself on every push to the main branch. The public address is the one configured in the Cloudflare Pages project (`*.pages.dev` domain or custom domain).
Backend (Supabase)	Supabase, organisation **it.team@h-farmschool.com's Org**, **Free** plan. Project address `https://gnahighyfoirnadhihez.supabase.co`	Dashboard: `https://supabase.com/dashboard` → project.
Database	Inside Supabase → *Table Editor* / *SQL Editor*	Tables listed in chapter 9.
Edge Functions	Supabase → *Edge Functions* (the code is also in `supabase/functions/` in the repo)	`submit-form` , `notify-powerautomate` , `send-auth-email` .
Edge Function secrets	Supabase → *Edge Functions* → *Secrets*	List in chapter 11.
Database secrets	Supabase → *Vault* (or `vault.secrets` via SQL)	Teams flow URL and platform address.
SQL migrations	`supabase/migrations/` in the repo (0001 → 0024)	Run by hand in the SQL Editor.
Automation flows	**Microsoft Power Automate** (H-FARM School tenant)	They receive the forms via webhook.
Flow documentation	`supabase/functions/notify-powerautomate/README.md` , `supabase/notifications/teams-task-assigned/`	Data schema and instructions.
Excel registries / photos	SharePoint / OneDrive (links in the HR and Boarding pages)	Written by the Power Automate flows.
Device inventory	Asset Manager `https://assetmanager.h-farm.com/`	The QR stickers on the devices point here.
Cameras	Cisco Meraki Vision `https://vision.meraki.com/`	Link from the Boarding section.

4. External platforms used

Service	What it is for	Where it is managed
GitHub	Keeps the source code and the history of changes.	github.com → HISIT-Team/management-platform

Service	What it is for	Where it is managed
Cloudflare Pages	Builds and publishes the site; applies the security headers (`public/_headers` file).	Cloudflare dashboard → Workers & Pages → platform project
Cloudflare Turnstile	Invisible "captcha" on the login, sign-up and password-recovery pages: blocks bots.	Cloudflare dashboard → Turnstile. The *site key* is in the site, the *secret key* is in Supabase (Auth → Attack Protection).
Supabase	The whole backend: authentication, database, Edge Functions, Vault.	supabase.com/dashboard
Supabase Auth	Email+password login, sign-up, password recovery, MFA (TOTP), sessions.	Supabase → Authentication
PostgreSQL (in Supabase)	The database with tables, RLS rules, functions and triggers.	Supabase → Table Editor / SQL Editor
pg_net (extension)	Lets the database call a webhook (Teams notification when a task is assigned).	Supabase → Database → Extensions
Supabase Vault	Encrypted safe for the secrets used by the database.	Supabase → Vault
pg_cron (optional)	Scheduling of periodic jobs (e.g. deleting old signatures). Not active yet.	Supabase → Database → Extensions
Microsoft Power Automate	Receives the forms (deliveries, onboarding, offboarding, rooms) and task notifications; writes to Excel, saves photos to OneDrive, sends recap emails and Teams messages.	make.powerautomate.com
Microsoft Teams	Receives the "a task was assigned to you" notifications.	Through Power Automate (Flow bot)
Microsoft Entra ID + Microsoft Graph	Sends the authentication emails from the **itsupport** mailbox (`send-auth-email` function).	Entra admin center → App registrations; Exchange Online (permission limited to that mailbox)
Exchange Online	Mailbox the login/sign-up/password emails are sent from (itsupport).	Exchange admin center
Resend	SMTP service used so far by Supabase for authentication emails. **To be retired** once the `send-auth-email` hook is active and working.	resend.com + Supabase → Authentication → Emails → SMTP
Google Fonts	The site typefaces (Plus Jakarta Sans, Fraunces).	Nothing to manage
esm.sh / npm	Sources of the libraries the Edge Functions load at deploy time.	Nothing to manage
H-FARM Asset Manager	Device inventory; the QR stickers contain the link to the device page (e.g. `.../hardware/25142/view`), the platform extracts the ID.	assetmanager.h-farm.com
SharePoint / OneDrive	Excel registries (onboarding, offboarding, rooms) and photos.	Microsoft 365
Cisco Meraki Vision	Cameras (link only).	vision.meraki.com

Main code libraries (in `package.json`)

Library	Use
Next.js 16 + **React 19** + **TypeScript**	The framework the site is built with.
@supabase/supabase-js	Connection to Supabase (login, data, functions).
jsqr	Reading QR codes from the camera (students and device stickers).
qrcode	Generating the student QR cards.
write-excel-file	Excel exports (history, budget, etc.).

5. File structure

```
management-platform/
├── package.json            ← list of libraries and commands (dev, build, lint)
├── next.config.ts          ← Next.js settings: "output: export" = static site
├── .env.example            ← template of the environment variables (no real values)
├── README.md               ← old readme (partly outdated: this guide prevails)
├── AGENTS.md / CLAUDE.md   ← notes for coding assistants
├── public/                 ← files published as they are
│   ├── _headers            ← HTTP security rules for Cloudflare (CSP etc.)
│   ├── robots.txt          ← forbids indexing by search engines
│   ├── site.webmanifest    ← data for "Add to Home Screen" (web app)
│   └── logo…, his-mp-…     ← logo and icons
├── src/                    ← SITE CODE (frontend)
│   ├── app/                ← one folder = one page (see table below)
│   │   ├── layout.tsx      ← structure shared by all pages (title, icons)
│   │   ├── page.tsx        ← the home (dashboard or welcome/sign-in page)
│   │   ├── globals.css     ← general style
│   │   ├── shell.css       ← style of the "frame" (side menu, top bar, dashboard)
│   │   ├── forms.css       ← style of the forms
│   │   ├── budget.css      ← style of budget and history pages
│   │   └── tasks.css       ← Task Manager style
│   ├── components/         ← "building blocks" reused in several pages
│   └── lib/                ← logic: Supabase connection, permissions, data
└── supabase/               ← BACKEND CODE (installed on Supabase by hand)
    ├── migrations/         ← 22 SQL files that build the database
    ├── functions/          ← the 3 Edge Functions
    ├── notifications/      ← instructions and templates for the Teams notification
    └── checks/             ← security_check.sql: database security checks
```

> **Caution:** the `node_modules/`, `.next/` and `out/` folders are generated automatically and must **not** be edited or uploaded to GitHub (they are excluded by `.gitignore`).

5.1 Pages (`src/app/`)

Each folder inside `src/app/` is an address of the site. Example: `src/app/device-history/` → `https://<site>/device-history`.

Address	What it contains	Who sees it (permission)
`/`	Dashboard (signed in) or welcome/sign-in page	Everyone
`/login`, `/signup`, `/reset-password`	Sign-in, sign-up, new password	Everyone (signed out)
`/mfa`	Entering / setting up the MFA code	Signed-in users
`/guest`	Waiting page for users without a role yet	Guest
`/profile`	My profile: data, picture, email, password, MFA, appearance	All signed-in users
`/it`	IT section hub	`it`
`/student-checkinout-hub`, `/modulo-student`	Student deliveries/returns (form)	`it.checkin_student`
`/employee-checkinout-hub`, `/modulo-employee`	Staff deliveries/returns (form)	`it.checkin_employee`
`/qr`	Student QR card generator	`it.checkin_student`
`/device-drafts`	Delivery/return drafts: open, sign, send	`it.checkin_student`
`/device-history`	Assignment history (search, edit, delete, export)	`it.history`
`/lost-accessories`	Lost accessories: report history, export	`it.lost_accessories`
`/lost-accessories/new`	Lost accessories report form	`it.lost_accessories`
`/it-registries-hub`	Response registries	`it.registries`
`/budget-management/venezia` · `/vicenza` · `/rosa`	Budget of the three schools	Owner/Super Admin/Admin only
`/budget-management`	Schools overview	Owner/Super Admin/Admin only
`/task-manager`	Task Manager	Owner/Super Admin/Admin only
`/hr`, `/employee-management-hub`, `/hr-registry-hub`	HR hub and registries	`hr`, `hr.registries`
`/onboarding`	New employee form	`hr.onboarding`
`/employee-management`	Offboarding form	`hr.offboarding`
`/boarding`, `/room-assignment-hub`, `/room-assignment`	Boarding and room assignment	`boarding`, `boarding.rooms`
`/student-office`	Student Office	`office`
`/backend`	Backend Management hub	Owner, Super Admin
`/user-management`	Users and roles	Owner, Super Admin
`/role-permissions`	Permissions matrix per role	Owner, Super Admin
`/audit-log`	Activity log	Owner, Super Admin

Address	What it contains	Who sees it (permission)
`/security-settings`	MFA per role, timeout, remembered devices	Owner, Super Admin

Usually each page has two files: `page.tsx` (the "envelope": title and access check) and `NameClient.tsx` (the actual content).

5.2 Components (`src/components/`)

File	What it does
`AppShell.tsx`	The frame: side menu, top bar, company switcher, mobile menu, automatic zoom on large screens.
`AuthGuard.tsx`	The "doorman": checks login, MFA and permissions before showing a page.
`Dashboard.tsx`	The home after login, different per company and role; company picker after login.
`DeviceCheckoutForm.tsx`	The delivery/return form (students and staff) with validation of every field.
`AssetIdField.tsx`	Asset ID field with scanning of the device QR sticker.
`QrScanner.tsx`	Student QR scanner (pre-fills the data).
`SignaturePad.tsx`	Signature box (requires a real stroke, not a tap).
`HubPage.tsx`	Card-based "hub" pages (IT, HR, Boarding...).
`IdleWatcher.tsx`	Signs the user out after the inactivity time set for the role.
`Turnstile.tsx`	Cloudflare's anti-bot check on the sign-in pages.
`Topbar` , `Header` , `Footer` , `Icon` , `useToast`	Graphic elements and pop-up messages.

5.3 Logic (`src/lib/`)

File	What it does
`supabase.ts`	Creates the Supabase connection using the environment variables.
`auth.ts`	Sign-in, sign-up, password recovery, profile reading, access checks, sending forms to `submit-form` .
`mfa.ts` , `trustedDevice.ts`	MFA and "remember this device".
`idle.ts`	Inactivity timeout.
`nav.ts`	The menu and the "who sees what" rule (including admin-only sections).
`permissions.ts`	List of permissions and reading/changing the matrix.
`companies.ts`	The three companies and which company each role sees.
`hubs.tsx`	Content of the hub pages (cards and links).
`deviceLog.ts` , `deviceExport.ts` , `deviceIcons.tsx`	Device history: saving, reading, editing, Excel export.
`deviceDrafts.ts`	Drafts of the device forms: save, load, list, delete.

File	What it does
`lostAccessories.ts`	Lost accessories: accessory list, sending a report, history.
`powerAutomate.ts`	Sending the device forms to `notify-powerautomate`.
`qrCamera.ts`	Camera and QR reading (high resolution, phone's native reader when available).
`image.ts`	Photo compression before sending.
`budgets.ts`, `dashboard.ts`	Budget lines, expenses, dashboard data.
`tasks.ts`	Task Manager.
`users.ts`, `profile.ts`	User management and personal profile.
`excel.ts`	Generic Excel export.
`uiPrefs.ts`	Appearance preferences (zoom, density).
`features.ts`	Switches: `SIGNUP_ENABLED` (public sign-up on/off), `PRIVACY_POLICY_URL`.

6. How the frontend works

6.1 Static site

`next.config.ts` contains `output: 'export'`: the build produces an `out/` folder with **only HTML, CSS and JavaScript files**. Cloudflare publishes them as they are. There is no platform server: every piece of data comes from Supabase, directly from the user's browser, with the user's access token.

6.2 Site environment variables

These values are inserted at build time (in **Cloudflare Pages → Settings → Environment variables**, and locally in the `.env.local` file, which does not go to GitHub):

Variable	Value	Secret?
`NEXT_PUBLIC_SUPABASE_URL`	`https://gnahighyfoirnadhihez.supabase.co`	No
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase *anon public* key (Project Settings → API)	No (public by design)
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	Turnstile widget *site key*	No

Anything starting with `NEXT_PUBLIC_` ends up in the browser: **never put secrets there**.

6.3 Security headers (`public/_headers`)

Cloudflare adds security rules to every page:

- **Content-Security-Policy (CSP):** the site may load scripts only from itself and Cloudflare Turnstile, fonts from Google Fonts, and may connect **only** to `gnahighyfoirnadhihez.supabase.co` and Turnstile. *If you ever change Supabase project, update this address here, or the site will not be able to connect.*
- **X-Frame-Options: DENY:** the site cannot be embedded in other pages.
- **Permissions-Policy:** allows only the camera (QR and photos); microphone and location are blocked.
- **HSTS:** forces HTTPS.

- **X-Robots-Tag / robots.txt:** the site is not indexed by Google.

6.4 Data stored in the browser

Some preferences stay in the user's browser (localStorage). They contain no sensitive data:

Key	Content
`sb-…-auth-token`	The Supabase session (access token).
`his:lastActivity`, `his:idleMinutes`	Last activity and timeout minutes of the role.
`his:mfaDevice`, `his:mfaRememberChoice`	Random token of "remember this device" (the database only holds its encrypted fingerprint).
`his:company`, `his:companyPick`	Selected company; request to choose after login.
`his:ui`	Appearance preferences.
`his:postLoginRedirect`	Page to reopen after login.

7. Roles, permissions and companies

7.1 Roles

Role	Description
owner	The highest level. Can do everything a Super Admin does, and in addition only an Owner can: give/remove the Owner role, edit or delete another Owner, reset their MFA, change the MFA rules of the Owner role. There is always at least one Owner.
superadmin	Full control of the platform, including Administration (users, permissions, log, security). Cannot touch Owners.
admin	Sees all operational sections of all companies, including Budget and Task Manager. Does not see Administration.
it	IT section (deliveries, history, registries) — H-IS Venezia.
hr	HR section — H-IS Venezia.
boarding	Boarding section — H-IS Venezia.
office	Student Office — H-IS Venezia.
parent	Legacy role, currently without sections.
office.hvi, **teachers.hvi**	H-IS Vicenza staff (currently no sections assigned).
office.hro, **teachers.hro**	H-IS Rosà staff (currently no sections assigned).
guest	Role of anyone who just signed up: sees nothing until an administrator assigns a role.

Roles are assigned from **Administration → Users and roles**. The role is stored in the `profiles` table (`role` column).

7.2 Permissions

Each role has a set of **permissions** (what it can open). They are managed from **Administration → Role permissions** (roles × permissions matrix). Owner and Super Admin always have everything; Guest has nothing.

Permission	What it opens
`it`	IT section (page and menu)
`it.checkin_student`	Student delivery/return forms, QR cards
`it.checkin_employee`	Staff delivery/return forms
`it.history`	Assignment history: read, search, edit, delete, export
`it.registries`	IT response registries
`it.lost_accessories`	Lost accessories: form, history, deletion
`it.budget` ADMIN	H-IS Venezia budget
`vi.budget` ADMIN	H-IS Vicenza budget
`ro.budget` ADMIN	H-IS Rosà budget
`it.tasks` ADMIN	Task Manager
`hr`, `hr.onboarding`, `hr.offboarding`, `hr.registries`	HR section and its functions
`boarding`, `boarding.rooms`	Boarding and room assignment
`office`	Student Office

ADMIN = **reserved to Owner, Super Admin and Admin** (migration 0022). These permissions **cannot be granted** to other roles, not even from the matrix: the database refuses.

Permissions are enforced **in two places**: in the pages (menu and access) and **in the database** (RLS rules and functions). So even someone querying Supabase directly would not get data they are not allowed to see.

7.3 The three companies

Company	Who sees it
H-IS Venezia	Legacy roles (it, hr, boarding, office, parent) + Owner/Super Admin/Admin
H-IS Vicenza	office.hvi, teachers.hvi + Owner/Super Admin/Admin
H-IS Rosà	office.hro, teachers.hro + Owner/Super Admin/Admin

Whoever can see more than one company (Owner, Super Admin, Admin) **picks the company after login** and can switch it from the side menu. Menu and dashboard show only the sections of the selected company. The company is a way to organise the platform; data protection comes from the permissions.

7.4 Access security

Feature	How it works	Where it is set
Password	Managed by Supabase Auth (never stored in clear text).	—
Turnstile captcha	On sign-in, sign-up, password recovery.	Cloudflare + Supabase Auth

Feature	How it works	Where it is set
MFA per role	For each role you decide whether the authenticator app is mandatory. Enforced in the database too: without the second factor, protected data stays closed.	Administration → Security
Remember this device	After the MFA code the browser can be "trusted" for N hours (default 24; 0 = off). The password is always required.	Administration → Security
Inactivity timeout	Automatic sign-out after N minutes without activity (5–1440, default 60), per role.	Administration → Security
Activity log	Every change of role, user, permission, MFA, history record, budget is logged.	Administration → Activity log
Public sign-up	On/off with `SIGNUP_ENABLED` in `src/lib/features.ts` (also turn it off in Supabase → Sign In / Providers). New users start as **guest**.	Code + Supabase

8. The backend: Supabase

Supabase brings together four services:

1. **Auth** — manages accounts (email+password), email confirmation, password recovery, MFA, sessions. - *Authentication → URL Configuration*: site address and allowed redirect addresses (`/login`, `/reset-password`, home page). - *Authentication → Emails*: email templates and SMTP (Resend). - *Authentication → Auth Hooks*: **Send Email** hook → `send-auth-email` function (once enabled, it replaces Resend). - *Authentication → Attack Protection*: Turnstile captcha (secret key). - *Authentication → Multi-Factor*: TOTP MFA enabled.
2. **Database (PostgreSQL)** — tables, RLS rules, functions and triggers (chapter 9).
3. **Edge Functions** — the three server-side programs (chapter 10).
4. **Vault** — secrets used by the database (chapter 11).

Free plan — things to know: - 500 MB of database: more than enough (photos do not go to the database, signatures are compressed). - **No automatic backups** on the Free plan: a periodic export is recommended (see chapter 15). - The project is **paused after about a week without use** (e.g. summer holidays): it is restored from the Supabase dashboard, no data is lost.

9. What the database stores

9.1 Tables

Table	What it contains	Who can access it
`profiles`	One profile per user: first name, last name, email (copy of the login one), **role**, profile picture (small, max 150 KB).	Each user their own; Owner/Super Admin all (through functions)
`auth.users` *(managed by Supabase)*	Sign-in accounts: email, encrypted password, MFA factors.	Supabase only

Table	What it contains	Who can access it
`student_device_log`	**Student assignment history**: date, delivery/return, student **email** (no names), school, MacBook ID, iPad ID, chargers and cables included (yes/no), who signed, **signature** (compressed image, max 60 KB), who recorded it.	`it.history` permission (read, edit, delete); insert with `it.checkin_student`
`device_form_drafts`	**Drafts** of the delivery/return forms: all form data and photos, **never the signature**. Temporary: deleted when the signed form is sent and after 14 days anyway.	`it.checkin_student` / `it.checkin_employee` permission
`lost_accessories_log`	**Lost accessories**: date, student or employee, **email** (no names, no parent emails), lost accessories, notes, who recorded it.	`it.lost_accessories` permission (read, insert, delete)
`budget_lines`	Each school's **budget lines**: name, description, colour, allocation.	The school's budget permission (admins only)
`budget_adjustments`	History of allocation changes (creation, increase/decrease, removal, reason, who).	Same as above
`it_budget_expenses`	**Expenses**: school, budget line, description, supplier, amount, date, notes, who entered it.	Same as above
`it_tasks`	**Tasks**: title, description, notes, group, assignees, status, priority, dates, estimated hours.	`it.tasks` permission (admins only)
`it_subtasks`	Sub-tasks.	Same as above
`it_task_groups`	Project groups.	Same as above
`it_task_members`	People who can be assigned tasks (name, email for Teams, colour). Managed from the Table Editor.	Same as above
`role_permissions`	The "role → permission" matrix.	Read by all signed-in users; changed only by Owner/Super Admin through a function
`mfa_role_policy`	For each role: MFA required yes/no, timeout minutes.	Changed only by Owner/Super Admin
`platform_settings`	General settings (e.g. "remember device" hours).	Changed only by Owner/Super Admin
`mfa_trusted_devices`	Remembered devices: only the **encrypted fingerprint** (SHA-256) of the token, expiry.	Database functions only
`audit_log`	**Activity log**: who, when, what (roles, users, permissions, MFA, history, budget). Cannot be changed from the platform.	Read only by Owner/Super Admin

9.2 What is **not** stored in the database

- **Students' first and last names**: by choice, the assignment history keeps only the email.
- **Names and parent emails in lost accessories**: they go only to the Power Automate flow; the history keeps email and accessories.
- **Device and damage photos**: they go only to Power Automate (saved to OneDrive/SharePoint by the flow).
- **Onboarding, offboarding, room assignment, staff device forms**: not saved in Supabase; they are forwarded to Power Automate, which records them in Excel/SharePoint.

- **Passwords**: managed by Supabase in encrypted form, never readable.

9.3 Main functions and triggers

Sensitive operations go through **database functions** that check permissions. The most important:

Function	What it is for
`has_permission`, `my_permissions`, `my_role`	Tell what the signed-in user can do (MFA rule included).
`admin_list_users`, `admin_update_user`, `admin_delete_user`, `admin_reset_mfa`	User management (Owner/Super Admin, with Owner protection).
`admin_set_role_permission`	Changes the permission matrix (refuses admin-only permissions).
`admin_set_mfa_policy`, `admin_set_idle_timeout`, `admin_set_mfa_remember_hours`	Security settings.
`mfa_ok`, `mfa_trust_device`, `mfa_forget_my_devices`	MFA and remembered devices.
`device_log_update`	Edits a history record (logging before/after in the activity log).
`budget_create_line`, `budget_adjust_line`, `budget_archive_line`	Budget lines and allocations.
`purge_device_log_signatures(months)`	Deletes signatures older than N months (GDPR retention).
`audit_write`	Writes to the activity log (used by the triggers).

Triggers (automatic actions):

Trigger	When it fires	What it does
`profiles_role_guard`	Profile created/changed	Forces `guest` on new users, blocks unauthorised role changes, protects Owners.
`profiles_audit`, `student_device_log_audit`	Changes/deletions	Write to the activity log.
`on_auth_user_email_changed`	Confirmed email change	Copies the new email into the profile.
`it_tasks_notify_assignees`	New task or new assignees	Calls the Power Automate flow → Teams message to the new assignees (via `pg_net`).
`it_tasks_sync_assignees`, `it_tasks_completed`, `*_touch`	Task changes	Keep assignees, completion date and modification date aligned.

10. The Edge Functions

They are in **Supabase → Edge Functions** and their code is in the repository under `supabase/functions/`. To update one: open the function in the dashboard, paste the new `index.ts` and click **Deploy** (or, from a terminal, `supabase functions deploy <name>`).

10.1 `submit-form`

- **What it does:** receives a form from the site, checks that the user is signed in, has the right permission and (if required) has passed MFA, then forwards the data to the matching Power Automate flow. The flow URLs are in secrets: **they never reach the browser**.
- **Who calls it:** `src/lib/auth.ts` → `submitForm()`.
- **Forms handled:**

Form type (`form_type`)	Permission required	Secret holding the flow URL
`onboarding`	`hr.onboarding`	`WEBHOOK_ONBOARDING`
`offboarding`	`hr.offboarding`	`WEBHOOK_OFFBOARDING`
`student_checkin`	`it.checkin_student`	`WEBHOOK_STUDENT_CHECKIN`
`student_checkout`	`it.checkin_student`	`WEBHOOK_STUDENT_CHECKOUT`
`employee_checkin`	`it.checkin_employee`	`WEBHOOK_EMPLOYEE_CHECKIN`
`employee_checkout`	`it.checkin_employee`	`WEBHOOK_EMPLOYEE_CHECKOUT`
`room`	`boarding.rooms`	`WEBHOOK_ROOM`
`lost_accessories`	`it.lost_accessories`	`WEBHOOK_LOST_ACCESSORIES`

- **Drafts:** when the form comes from a draft (`draft_id`), once the flow has accepted the form the function **deletes the draft** from the database.
- **Limits:** maximum request 16 MB (photos are already compressed in the browser).
- **Optional secrets:** `ALLOWED_ORIGINS` (allowed domains), `TURNSTILE_SECRET` (**do not set it**: the forms do not send the captcha yet and would be rejected).

10.2 `notify-powerautomate`

- **What it does:** after every device delivery/return, sends the Power Automate flow a complete summary (data, devices, IDs, chargers and cables, signature as an image, photos) for Excel, OneDrive and the recap email. Accepts only users with the `it.checkin_student` / `it.checkin_employee` permission and MFA passed if required.
- **Who calls it:** `src/lib/powerAutomate.ts`.
- **Secrets:** `POWER_AUTOMATE_WEBHOOK_URL`; optional `ALLOWED_ORIGINS`.
- **Limit:** 30 MB per request.
- **Flow documentation:** `supabase/functions/notify-powerautomate/README.md` (available fields), `sample-payload.json`, `request-schema.json`, `email-recap.html` (email template).

> A student delivery therefore triggers **three actions**: (1) `submit-form` → `WEBHOOK_STUDENT_CHECKOUT` flow; (2) a row in the **history** (`student_device_log`); (3) `notify-powerautomate` → recap flow. If (2) or (3) fail the user sees a warning, but the form still counts as sent.

10.3 `send-auth-email`

- **What it does:** it is the Supabase Auth **"Send Email" hook**. Every time Supabase must send an authentication email (sign-up confirmation, password recovery, email change, invite, magic link, codes), it calls this function, which sends it **from the itsupport mailbox** through **Microsoft Graph**, with the platform's look and in Italian. It replaces Resend.

- **Who calls it:** Supabase Auth (not the site). The request is signed: the function refuses anyone without the correct signature.
- **Mandatory setting: Verify JWT turned off** (Supabase calls it, not a user).
- **Secrets:** `SEND_EMAIL_HOOK_SECRET`, `MS_TENANT_ID`, `MS_CLIENT_ID`, `MS_CLIENT_SECRET`, `MAIL_FROM` (e.g. `itsupport@h-farmschool.com`), optional `MAIL_FROM_NAME`.
- **Microsoft permissions:** app registered in Entra "HIS Management Platform – Mail" **without** `Mail.Send` under API permissions; the send permission is granted in Exchange Online with *RBAC for Applications*, **limited to the itsupport mailbox only**.
- **Full instructions:** `supabase/functions/send-auth-email/README.md`.
- **Status:** ready; enabled from *Authentication → Auth Hooks → Send Email hook*. To go back to Resend, just disable the hook.
- **Expiry:** the Entra app *client secret* expires (max 24 months): renew it in time and update `MS_CLIENT_SECRET`.

11. Secrets and keys: where they are and what they are for

> This table says **where** the values are kept. The real values are **not** in this guide nor in the code.

11.1 In the site (Cloudflare Pages → Environment variables)

Name	What it is for	Confidential?
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project address	No
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase public key	No
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`	Captcha public key	No

11.2 Supabase → Edge Functions → Secrets

Name	Used by	What it is for
`WEBHOOK_ONBOARDING`	submit-form	Onboarding flow URL
`WEBHOOK_OFFBOARDING`	submit-form	Offboarding flow URL
`WEBHOOK_STUDENT_CHECKIN`	submit-form	Student return flow URL
`WEBHOOK_STUDENT_CHECKOUT`	submit-form	Student delivery flow URL
`WEBHOOK_EMPLOYEE_CHECKIN`	submit-form	Staff return flow URL
`WEBHOOK_EMPLOYEE_CHECKOUT`	submit-form	Staff delivery flow URL
`WEBHOOK_ROOM`	submit-form	Room assignment flow URL
`WEBHOOK_LOST_ACCESSORIES`	submit-form	Lost accessories flow URL (instructions in `supabase/notifications/lost-accessories/`)
`POWER_AUTOMATE_WEBHOOK_URL`	notify-powerautomate	Device recap flow URL
`ALLOWED_ORIGINS` *(optional)*	submit-form, notify-powerautomate	Site domains allowed to call the functions

Name	Used by	What it is for
`TURNSTILE_SECRET` *(do not set for now)*	submit-form	Captcha on forms
`SEND_EMAIL_HOOK_SECRET`	send-auth-email	Email hook signature (from Authentication → Auth Hooks)
`MS_TENANT_ID`	send-auth-email	Microsoft tenant ID
`MS_CLIENT_ID`	send-auth-email	ID of the "Mail" Entra app
`MS_CLIENT_SECRET`	send-auth-email	Secret of the "Mail" Entra app (**expires**)
`MAIL_FROM` / `MAIL_FROM_NAME`	send-auth-email	Sender mailbox and display name
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`	all	**Inserted automatically by Supabase**: do not create them.

Secrets no longer used (can be deleted if present): `WEBHOOK_MEDICINE`, `WEBHOOK_DIET`, `WEBHOOK_PURCHASE_VICENZA`.

11.3 Supabase → Vault (database secrets)

Name	What it is for
`task_notify_webhook_url`	URL of the Power Automate flow that sends the Teams notification for tasks
`platform_url`	Platform address (for the link in the Teams card)

11.4 Other secrets

Where	What
Supabase → Project Settings → API	**Service role key** (top secret: never in the site) and anon key
Supabase → Authentication → Attack Protection	Cloudflare Turnstile secret key
Supabase → Authentication → Emails → SMTP	Resend SMTP credentials (while in use)
Microsoft Entra → App registrations → "HIS Management Platform – Mail"	Client secret (copied into `MS_CLIENT_SECRET`)
Power Automate	The HTTP trigger URLs of the flows (whoever knows them can send data to the flow: treat them as passwords)
GitHub / Cloudflare / Supabase	The sign-in accounts of the three consoles (use team accounts, with MFA)

If a secret falls into the wrong hands: regenerate it in the originating service (new flow URL, new client secret, new key), update the matching secret and, for Edge Functions, no redeploy is needed (secrets are read on every call).

12. How the main flows work

12.1 Sign-in

1. The user opens the site and enters email and password; Turnstile checks it is not a bot.

2. Supabase Auth checks the credentials and returns a token.
3. If the role requires MFA, the `/mfa` page asks for the app code (unless the device is "remembered").
4. The platform reads profile, role and permissions; whoever sees more than one company picks the company.
5. After N minutes of inactivity the user is signed out.

12.2 Delivering or returning a device (student)

1. IT opens *Student deliveries* → Check-out (delivery) or Check-in (return).
2. (Optional) scans the student QR card: the data fills in by itself.
3. Selects the devices; for MacBook/iPad/Accessories types or scans the **Asset ID** from the sticker (the Asset Manager URL, e.g. `.../hardware/25142/view` → `25142`).
4. For MacBook and iPad adds at least 2 photos (front and back) and any damage photos.
5. Chooses who signs and collects the signature. **Every field is mandatory** (except the second parent email).
6. On submit: `submit-form` → flow; a row in the **history** (`student_device_log`); `notify-powerautomate` → Excel, OneDrive, recap email.

12.2-bis Drafts (prepare first, sign later)

1. In the delivery/return form scan the QR and fill devices, Asset IDs and photos, then press **Save as draft**. The form is cleared, ready for the next student.
2. Drafts are in *IT → Bozze consegne* (`/device-drafts`), visible to the whole IT team from any device.
3. When the student is there: **Apri e fai firmare** (open and sign) → the pre-filled form opens → signature → **Submit form**.
4. On submit the draft is **deleted**: by the `submit-form` Edge Function (server side, as soon as the flow accepted the form) and, as a safety net, by the browser, which checks it no longer exists.
5. A draft that is never sent expires after **14 days** and is deleted when someone opens the Drafts page (or nightly with pg_cron, optional). The signature is never saved in a draft.

12.3 Assignment history

Search by email or ID, filters, current device status ("who has what"), charger/cable badges, **pencil** to correct a record, **bin** to delete it, Excel export. Every edit/deletion goes into the activity log.

12.3-bis Lost accessories

1. IT opens *Accessori smarriti* → **Nuova segnalazione** (new report).
2. Chooses **Student** or **Employee**. For students the QR card can be scanned (fills first and last name, email, parent emails, school).
3. Fills first name, last name, email (for students also parent email 1; email 2 is optional).
4. Selects one or more accessories: Mac power adapter, iPad power adapter, MagSafe cable for Mac, USB Type-C cable for iPad; optional notes.
5. On submit: `submit-form` → `WEBHOOK_LOST_ACCESSORIES` flow (with all the data) and a row in `lost_accessories_log` (**without names or parent emails**).
6. The history shows counts per accessory, search by email, filters, Excel export and deletion (logged in the activity log).

12.4 Budget

For each school: budget lines with allocation, expenses, remaining amount. *New budget line* button; *Budget* button on each line to add/remove allocation with a reason; a line without expenses can be removed.

Everything is tracked in `budget_adjustments` and in the activity log.

12.5 Task Manager and Teams

When a task is created or gets new assignees, the database (trigger + `pg_net`) calls the Power Automate flow, which sends a Teams card only to the **new** assignees (not to whoever made the change). An email in `it_task_members` is required.

12.6 Authentication emails

Sign-up, password recovery, email change: Supabase prepares the link and sends the email through **Resend (SMTP)** or, with the hook enabled, through `send-auth-email` → **Microsoft Graph → itsupport mailbox**.

13. Database migrations

Migrations are in `supabase/migrations/` and must be run **once, in order**, in the Supabase **SQL Editor** (open the file, copy everything, paste, *Run*). They are **re-runnable**: running one again breaks nothing, but some notes at the top of the files say which migrations to run again afterwards.

No.	Content
0001	IT budget expenses table
0002	Expenses split per school
0003	Task Manager (tasks, sub-tasks, groups, members)
0004	Task notes
0005	Student device history (`student_device_log`)
0006	Deleting history records
0007	New users as `guest`; only Super Admin changes roles
0008	Super Admin role and User Management
0009	First and last name from sign-up to profile
0010	Fix for role changes by Super Admin
0011	Several assignees per task
0012	Profile picture and email change
0013	Teams notification on task assignment (Vault + pg_net)
0014	Security and GDPR: activity log, protected email, signature purge
0015	MFA required per role
0016	Timeout per role, "remember device", admin sections
0017	Owner role and permissions per role
0018	Three companies, new roles, budget lines in the database
0019	Removal of the Purchases section
0020	Editing history records
0021	Chargers and cables in the history

No.	Content
0022	Budget and Task Manager for Owner, Super Admin, Admin only
0023	Lost accessories (permission, history table without names)
0024	Drafts of the delivery/return forms

Rebuilding the database from scratch (e.g. new Supabase project): run 0001 → 0024 in order, then create the first Owner (see 15.1), insert the Vault secrets (0013) and recreate the Edge Function secrets.

The file `supabase/checks/security_check.sql` contains security check queries (RLS enabled, policies present) to run in the SQL Editor.

14. How to publish a change

14.1 Change to the site (frontend)

1. Edit the files in the local copy `~/Downloads/management-platform/management-platform`.
2. (Optional) try it locally from the Terminal: `npm install (only the first time or when libraries change)` `npm run dev (test site at http://localhost:3000)`
3. Publish: `cd ~/Downloads/management-platform/management-platform git add -A git commit -m "Description of the change" git push`
4. Cloudflare Pages sees the push, runs the build (`npm run build`, which generates the `out/` folder) and publishes. Within a few minutes the new version is online. **Cloudflare → project → Deployments** shows the result; if there is an error, read the log: the previous version stays online.

14.2 Change to the database

1. Create (or receive) a new numbered migration file, e.g. `0023_….sql`, in `supabase/migrations/`.
2. Run it in the Supabase **SQL Editor**.
3. Also `git push`, so the file stays in the repository.

> Recommended order when a change touches both database and site: **migration first, then push**.

14.3 Change to an Edge Function

1. Update the file `supabase/functions/<name>/index.ts`.
2. Supabase → Edge Functions → → paste the code → **Deploy** (keeping the *Verify JWT* setting given in chapter 10).
3. `git push` to keep the code aligned.

14.4 Change to a Power Automate flow

Done directly in Power Automate. If the trigger **URL** changes, update the matching secret (chapter 11).

15. Maintenance and frequent operations

15.1 Users and roles

- **New employee:** signs up from `/signup` (starts as *guest*) → an Owner/Super Admin assigns the role from *Users and roles*.
- **First Owner** (once only, in the SQL Editor): `sql update public.profiles set role = 'owner' where id = (select id from auth.users where email = 'name.surname@h-farmschool.com');`
- **A user lost their MFA phone:** *Users and roles* → *Reset MFA*.
- **The only Owner lost their MFA phone** (SQL Editor): `sql delete from auth.mfa_factors where user_id = (select id from auth.users where email = 'name.surname@h-farmschool.com');`
- **Employee leaving:** set the role to *guest* or delete the user from *Users and roles*.

15.2 Task Manager members

Supabase → Table Editor → `it_task_members`: add people, emails (for Teams), deactivate with `active = false`.

15.3 Data retention (GDPR)

- Signatures older than N months: `select public.purge_device_log_signatures(24);` (example: 24 months). It can be scheduled with pg_cron (instructions at the end of migration 0014).
- The activity log keeps the history of changes: do not alter it.

15.4 Backup (Free plan)

The Free plan **does not make automatic backups** (daily backups come only with paid plans). Recommended at least once a month:

- export the main tables from the **Table Editor** (*Export → CSV*): `profiles`, `student_device_log`, `budget_lines`, `budget_adjustments`, `it_budget_expenses`, `it_tasks`, `it_subtasks`, `it_task_groups`, `it_task_members`, `role_permissions`, `mfa_role_policy`, `audit_log`;
- or, for Terminal users, a full backup with `pg_dump` using the connection string (Project Settings → Database).

Keep the files in a safe, restricted place (they contain emails and signatures), for example an IT team SharePoint folder.

15.5 Deadlines to remember

What	When
Client secret of the "Mail" Entra app	At the chosen expiry (max 24 months)
Supabase Free project pause	After ~7 days without use (holidays)
Domain / service account renewals	According to the contracts

16. Troubleshooting

Problem	What to check
The site does not open or shows an old version	Cloudflare → Deployments: did the last build succeed? Clear the browser cache.
"Supabase unreachable" / empty pages	Is the Supabase project paused? (Supabase dashboard → *Restore*). Are the `NEXT_PUBLIC_SUPABASE_*` variables on Cloudflare correct?
A user does not see a section	Role in *Users and roles*; permissions in *Role permissions*; selected company; MFA passed if required. Budget and Task Manager are admin-only.
"Prima attiva la tua MFA" (set up your MFA first)	The user must set up the authenticator app from the Profile.
A form is not sent	Supabase → Edge Functions → *submit-form* → **Logs**. Is the `WEBHOOK_*` secret there? Is the Power Automate flow on?
The device recap email is missing	*notify-powerautomate* logs; flow run history in Power Automate.
Sign-up/password email does not arrive	With the hook on: *send-auth-email* logs (Microsoft 403 errors = Exchange permission not active yet, wait up to 2 hours). With the hook off: SMTP/Resend settings. Check spam too.
The task Teams notification does not arrive	Person's email in `it_task_members`; Vault secrets; in the SQL Editor `select * from net._http_response order by created desc limit 5;`
The QR scanner does not read	Browser camera permission; sticker inside the square at 10–15 cm; use the flashlight. If "QR non riconosciuto" appears, the message shows the text that was read.
Error after re-running an old migration	Read the *NOTE: … re-run …* lines at the top of the file and re-run the migrations listed (usually 0016, 0017, 0018, 0021, 0022).

17. Final notes

- The repository's `README.md` describes an earlier phase of the project (e.g. Netlify, medicine and diet forms later removed): **this guide prevails**.
- Every important change to the platform should come with an update to this guide.
- Technical contact: H-FARM International School IT team.